

Rapport Technique — BadWallet Web Dashboard

Examen Design Pattern / UML — L3 S2 2026

Marie Diagne

1. Choix d'architecture

L'application a été développée avec Angular 22 en Standalone Components, sans NgModules. Cette approche, recommandée depuis Angular 14, favorise la séparation des responsabilités, le tree-shaking et le lazy loading automatique de chaque composant.

Structure en couches

- core/ : services (WalletApiService, BillingApiService, AuthService, NotificationService), intercepteur HTTP et guards de routes
- shared/ : composants réutilisables (Header, Toast, Loader) et pipe XofPipe pour le formatage de la devise XOF
- features/ : composants métier par espace (agent/, client/, login/), tous chargés en lazy loading

State Management avec Angular Signals

Le solde courant est stocké dans un Signal (signal<number>(0)) du WalletApiService. Chaque opération financière (dépot, retrait, transfert) met à jour ce signal via tap(). Le Header affiche en temps réel le solde sans rechargement de page.

2. Fonctionnalités implémentées

Fonctionnalité	Endpoint	Espace
Listing portefeuilles page	GET /api/wallets	Agent
Création portefeuille	POST /api/wallets	Agent
Recherche par téléphone	GET /api/wallets/{phone}	Agent
Dépot / Retrait (frais 1%)	POST /api/wallets/{id}/deposit	Agent
Dashboard + solde temps réel	GET /api/wallets/{phone}/balance	Client
Transfert avec validation	POST /api/wallets/transfer	Client
Paiement factures (multi-select)	POST /api/wallets/pay-factures	Client
Historique avec filtres	GET /api/wallets/{phone}/transactions	Client
Factures impayées du mois	GET /api/external/factures/...	Client

3. Difficultés rencontrées et solutions

CORS entre Angular et Spring Boot

Angular (port 4200) appelle Spring Boot (port 8080), ce qui déclenche une erreur CORS ("Access-Control-Allow-Origin" absent). Solution : ajout d'une classe CorsConfig dans badwallet-api autorisant toutes les origines en développement.

SSR et proxy Angular

Angular 22 active le Server-Side Rendering par défaut lors de la création du projet, ce qui empêche le proxy de fonctionner correctement. Solution retenue : configuration CORS côté API plutôt que proxy, compatible avec SSR.

Synchronisation entre les deux APIs

Le payment-service gère des factures liées aux codes portefeuille (WLT-XXXXXXX) générés par badwallet-api. Les deux seeders utilisent le même format de code et le seed du payment-service doit être lancé après celui de badwallet-api pour garantir la cohérence des données.

4. Organisation GitFlow

Chaque fonctionnalité a été développée dans une branche feature/* dédiée (13 branches au total), puis mergée dans develop via --no-ff pour conserver l'historique détaillé des contributions. La branche main contient la version finale de livraison, taguée pour le rendu.

Depot GitHub : <https://github.com/MarieDiagne/badwallet-dashboard>